MIT 8103 Advanced Database Systems - Portfolio Documentation

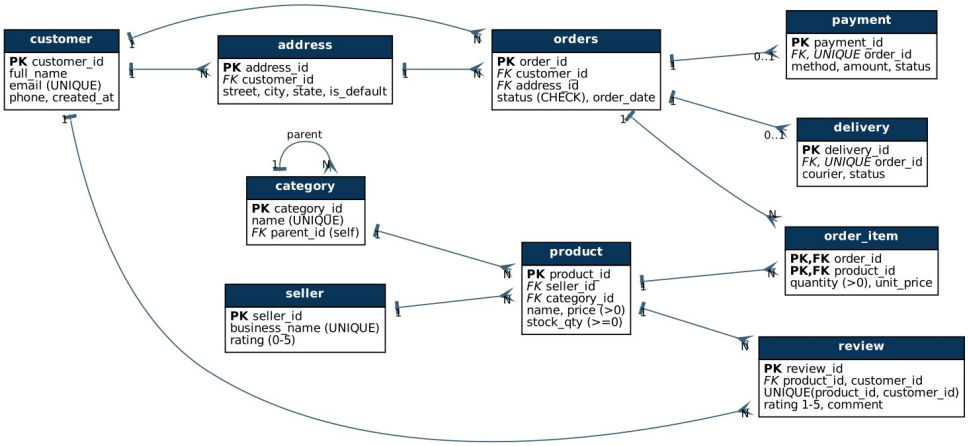
Student: Zainab Abubakar Bakabe **Matric No.:** 2026/A/MIT/0791 **Student ID:** 301929341 **GitHub:** https://github.com/zainababubakarbakabe-alt/advanced-database-portfolio (private; assessor added as collaborator)

1. Selected organisation

NaijaCart is a *simulated* Nigerian multi-seller e-commerce marketplace. **Users of the database:** customers, sellers, dispatch staff, finance staff and analysts. **Major data:** customers and addresses, sellers, a category tree, products and stock, orders and order items, payments (card, transfer, USSD, wallet), deliveries and reviews. **Key operations:** browse the catalogue, place an order and reduce stock, take payment, track delivery, review products, report sales. Test data: 50,000 customers, 20,000 products, 200,000 orders and 600,000 order items, generated with a fixed seed so every run is identical. The same case study runs through all five portfolios on PostgreSQL 16 and Redis 7; `run_all.sh` rebuilds everything in about two minutes.

2. Portfolio 1 - Database design and modelling

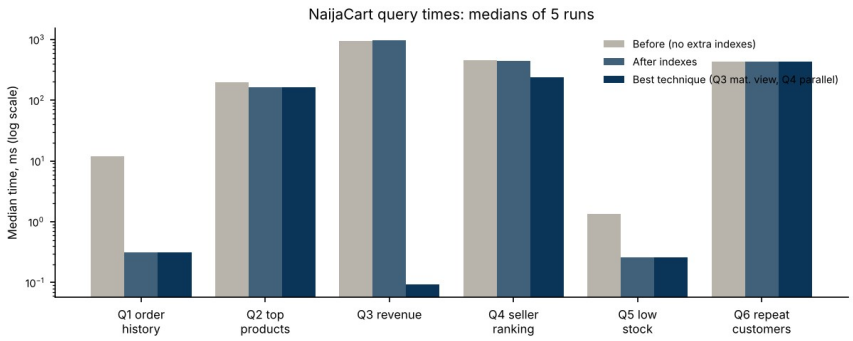
The 11-table schema is in **BCNF**: every functional dependency's determinant is a candidate key (listed in the README). Normalising a spreadsheet-style order sheet turned repeating product columns into `order_item` rows (1NF), moved product facts out of the composite-key table (2NF) and moved customer, seller and category facts into their own tables (3NF). Business rules are enforced by the database, not just the application: `CHECK (stock_qty >= 0)`, positive prices and quantities, status lists, one payment and one delivery per order, and one review per customer per product. `order_item.unit_price` is a deliberate denormalisation that preserves the price paid. Four roles implement least privilege; an analyst login could read 600,000 rows through an anonymised view but was refused access to customer emails and could not delete orders (NDPA 2023 data minimisation).



NaijaCart ER diagram (crow's foot notation)

3. Portfolio 2 - Query processing and optimisation

Seven queries (joins, CTEs, a window function, aggregations) were measured with `EXPLAIN (ANALYZE, BUFFERS)` and timed as the **median of five runs**, before and after optimisation. Indexes chosen from the plans gave large gains only on **selective** queries: customer order history **12.0 to 0.31 ms (38.6x)**, replacing a sequential scan of 200,000 rows with an index scan; low-stock alert 1.35 to 0.26 ms (5.1x) with a covering index. Full-scan analytics did not improve with indexes, so other techniques were tested: a **materialized view** reduced the revenue report from 952 ms to 0.09 ms (at a refresh cost of 567 ms), and **parallel query** made Q3 and Q4 1.8x faster. The repeat-customer query's plan showed an 11 MB disk sort; raising `work_mem` moved the sort into memory but did **not** reduce the time, proving the join, not the sort, was the bottleneck. Costs were measured too: the new indexes use 16.7 MB and made 20,000 order inserts 2.8x slower (15.3 to 42.3 ms).



Median query times before and after optimisation (log scale)

4. Portfolio 3 - Transactions and concurrency

`place_order()` performs the order, stock decrement, items, payment and stock log as one atomic transaction. An order for 10 units when 3 remained violated the `CHECK` constraint and **rolled back completely** (order count unchanged); a `SAVEPOINT` undid one of two updates. Two concurrent sessions reproduced a **lost update**: both bought the last unit (stock oversold). Pessimistic locking (`FOR UPDATE`) and an atomic conditional `UPDATE` each fixed it, selling exactly one unit. READ COMMITTED showed a non-repeatable read (10,000 then 12,000) that REPEATABLE READ prevented; SERIALIZABLE aborted a write-skew transaction, and a retry loop then re-ran it correctly. Opposite lock order caused `deadlock detected`; a consistent lock order removed it. Optimistic locking (a version column) rejected a stale admin edit, and an MVCC test showed a reader seeing the old committed price without waiting while a writer held an uncommitted change. Each technique is matched to the NaijaCart situation it suits.

5. Portfolio 4 - NoSQL and advanced data models

Orders, payments and stock stay relational because they need ACID guarantees; other workloads use the model that fits them (polyglot persistence). **Document:** the catalogue as 20,000 JSONB documents with category-specific attributes and embedded latest reviews; a GIN index cut an attribute search from 6.2 to 0.63 ms, and loading 2,000 product pages took **24 ms as documents versus 106 ms relationally (4.4x)**. **Key-value (Redis):** carts as hashes with a 7-day TTL, sessions with a 30-minute expiry, an atomic `DECR` flash-sale counter (the fourth buyer for three units got -1 and was rejected) and a sorted-set best-seller board. **Graph:** a Cypher design for “customers who bought this also bought”. Embedding-versus-referencing and the consistency of each store are justified in the README. The MongoDB script is provided with run instructions.

6. Portfolio 5 - Distributed and cloud database

A primary and a streaming hot standby were built with `pg_basebackup`. Asynchronous replication applied writes in about 0.1 s and the replica rejected writes. Switching to **synchronous** replication and stopping the standby **blocked the next commit** (`wait_event = SyncRep`), demonstrating CAP’s choice of consistency over availability; the primary was then crashed and the replica **promoted**, and service continued. Orders were **hash-sharded** on `customer_id` across two nodes with `postgres_fdw` (10,098 and 9,902 rows); a per-customer query was pruned to one shard, while a global report had to scan both. The cloud design uses synchronous commit to any one of two standbys for payments, asynchronous replicas for browsing and reports, managed Redis, point-in-time recovery, and data kept in Nigeria where possible to simplify NDPA cross-border rules.

7. Individual technical reflection

The biggest lesson was that **measurement beats assumption**. I expected indexes to speed up every slow query, but the plans showed they only help selective lookups: order history became 38.6x faster while the revenue report did not change, because it needs almost every row. The same lesson came from the repeat-customer query, whose plan showed an 11 MB disk sort; when I gave it enough memory the sort moved into RAM but the query was no faster, so the real bottleneck was the join. A diagnosis is only a hypothesis until it is tested.

Concurrency bugs were the hardest to see. With one session, the read-then-write stock update looked correct; it only oversold when two sessions ran at the same moment, which I had to force with deliberate delays. I also learnt that SERIALIZABLE is only safe with a retry loop, and that a synchronous commit cannot be stopped by `statement_timeout` - my first test hung until I cancelled the waiting session explicitly.

I also found errors in my own work. My first phone data never contained a 128 GB model because of a modulo pattern, so a correct query returned zero rows. A benchmark loop called `random()` inside `WHERE`, which forced full table scans and made the document model look slower than it was. A shell-quoting mistake silently enabled parallel query and distorted a whole set of timings. Fixing these is why every result is now re-run from one script with a fixed seed. Design choices I would defend: the `CHECK (stock_qty >= 0)` constraint as the last line of defence against overselling, `customer_id` as the shard key because most queries are per customer, and keeping money in ACID PostgreSQL while Redis holds only temporary data. Next, I would run the MongoDB and Neo4j designs on real servers and test on a machine with more CPUs.

8. References and AI-use declaration

- Abadi, D. (2012) 'Consistency tradeoffs in modern distributed database system design', *Computer*, 45(2), pp. 37-42.
- Brewer, E. (2012) 'CAP twelve years later: how the "rules" have changed', *Computer*, 45(2), pp. 23-29.
- Federal Republic of Nigeria (2023) *Nigeria Data Protection Act 2023*.
- Kleppmann, M. (2017) *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly.
- MongoDB Inc. (n.d.) *MongoDB Manual*. Available at: https://www.mongodb.com/docs/manual/.
- Ports, D.R.K. and Grittner, K. (2012) 'Serializable snapshot isolation in PostgreSQL', *Proceedings of the VLDB Endowment*, 5(12), pp. 1850-1861.
- PostgreSQL Global Development Group (n.d.) *PostgreSQL 16 Documentation*. Available at: https://www.postgresql.org/docs/16/.
- Redis Ltd (n.d.) *Redis Documentation*. Available at: https://redis.io/docs/.
- Silberschatz, A., Korth, H.F. and Sudarshan, S. (2019) *Database System Concepts*. 7th edn. New York: McGraw-Hill.

AI-use declaration. (i) **Tool:** Claude (Anthropic), used through claude.ai. (ii) **Used for:** choosing and scoping the case study; designing the schema; writing the SQL, shell and Python scripts; running them on PostgreSQL 16 and Redis 7 to produce the outputs in `results/`; reviewing the work against the marking rubric; and drafting the READMEs and this document, including a first draft of this reflection. (iii) **Review and verification:** every script was executed and its raw output saved in `results/`; every figure quoted in this document is taken from those files; timings are medians of five runs on data generated with a fixed seed, and two full rebuilds produced identical data; errors found during testing (listed in section 7) were corrected and the affected scripts re-run. The MongoDB script and the Neo4j design were not executed.

Appendix - Evidence (excerpts from the `results/` files in the repository)

Evidence 1 - Portfolio 2: execution plans and timings

```
# excerpt from: portfolio-2-query-optimisation/results/before_q1_customer_order_history.txt
                    ->  Seq Scan on orders o  (cost=0.00..4287.00 rows=5 width=24) (actual time=1.550..21.139…
 Execution Time: 21.505 ms
-- median of 5 further runs: 12.017 ms

# excerpt from: portfolio-2-query-optimisation/results/after_q1_customer_order_history.txt
                         ->  Bitmap Index Scan on idx_orders_customer_date  (cost=0.00..4.46 rows=5 width=0)…
 Execution Time: 0.382 ms
-- median of 5 further runs: 0.311 ms

# full file: portfolio-2-query-optimisation/results/timing_summary.txt
query                              before (ms)     after (ms)  speed-up
q1_customer_order_history               12.017          0.311     38.6x
q2_top_products_90_days                196.018        161.887      1.2x
q3_revenue_by_category                 952.062        959.476      1.0x
q4_seller_rank_window                  457.335        446.769      1.0x
q5_low_stock_pending                     1.345          0.263      5.1x
q6_cte_repeat_customers                429.038        432.673      1.0x
q6b_repeat_customers_rewritten         424.122        435.151      1.0x

# excerpt from: portfolio-2-query-optimisation/results/06_parallel_query_output.txt
q3_revenue_by_category     workers=0  launched=0   median= 1062.321 ms
q3_revenue_by_category     workers=2  launched=2   median=  605.865 ms
q3_revenue_by_category     workers=4  launched=2   median=  565.854 ms
q4_seller_rank_window      workers=0  launched=0   median=  435.141 ms
q4_seller_rank_window      workers=2  launched=2   median=  239.165 ms
q4_seller_rank_window      workers=4  launched=2   median=  251.333 ms
```

Evidence 1: Portfolio 2 plans and timings

Evidence 2 - Portfolio 3: lost update, isolation, deadlock, retry

```
# excerpt from: portfolio-3-transactions-concurrency/results/03_lost_update_output.txt
=== B1. BROKEN pattern: read stock in app, then write back (read-modify-write)
session B read stock = 1
session A read stock = 1
units sold (stock_log) = 2, stock now = 0
-> 2 customers were told they bought the last unit: OVERSOLD (lost update).
=== B2. FIX 1: pessimistic locking with SELECT ... FOR UPDATE
session A got lock, stock = 1
session A units sold: 1
session B got lock, stock = 0
session B units sold: 0
final stock = 0
=== B3. FIX 2: atomic conditional UPDATE + CHECK constraint (no explicit lock needed)
session A units sold: 1
session B units sold: 0
final stock = 0

# excerpt from: portfolio-3-transactions-concurrency/results/04_isolation_levels_output.txt
=== Isolation level: READ COMMITTED
T1 first read  price = 10000.00
T1 second read price = 12000.00
=== Isolation level: REPEATABLE READ
T1 first read  price = 10000.00
T1 second read price = 10000.00
ERROR:  could not serialize access due to read/write dependencies among transactions

# excerpt from: portfolio-3-transactions-concurrency/results/05_deadlock_output.txt
ERROR:  deadlock detected
-> No deadlock: T2 simply waited for T1 to finish.

# full file: portfolio-3-transactions-concurrency/results/06_serializable_retry_output.txt
TB attempt 1: committed
TA attempt 1: serialization failure (40001) -> retrying
TA attempt 2: committed
final: 401=1, 402=0 (rule holds: total >= 1)
```

Evidence 2: Portfolio 3 concurrency demonstrations

Evidence 3 - Portfolio 4: document model and Redis

```
# excerpt from: portfolio-4-nosql/results/02_document_queries_output.txt
 Seq Scan on product_catalog (actual rows=277 loops=1)
 Execution Time: 6.196 ms
   ->  Bitmap Index Scan on idx_catalog_doc_gin (actual rows=277 loops=1)
 Execution Time: 0.625 ms

# excerpt from: portfolio-4-nosql/results/06_relational_vs_document_output.txt
relational: Time: 106.333 ms
document:   Time: 24.020 ms

# excerpt from: portfolio-4-nosql/results/03_redis_output.txt
    2
    3
    1
    3
    1
    604800
    1800
# 3) Flash sale: 3 units of product 9001. DECR is atomic, so concurrent buyers cannot oversell.
127.0.0.1:6379> SET flash:stock:9001 3
127.0.0.1:6379> DECR flash:stock:9001
    2
127.0.0.1:6379> DECR flash:stock:9001
    1
127.0.0.1:6379> DECR flash:stock:9001
```

Evidence 3: Portfolio 4 document model and Redis

Evidence 4 - Portfolio 5: replication, synchronous commit, failover, sharding

```
# excerpt from: portfolio-5-distributed-cloud/results/01_replication_output.txt
 127.0.0.1   | streaming | async      | 0/19000000 | 0/19000000
primary in_recovery=false
replica price  = 99999.00
ERROR:  cannot execute UPDATE in a read-only transaction
replica replay delay    = 00:00:00.09633
sync_state now = sync
after 3 s the committing session is still waiting: state=active, wait_event=SyncRep
WARNING:  canceling wait for synchronous replication due to user request
new primary in_recovery=false
write accepted on promoted node

# excerpt from: portfolio-5-distributed-cloud/results/02_sharding_output.txt
 shard 1 (node A) | 10098
 shard 2 (node B) |  9902
 Seq Scan on orders_s1 orders
         ->  Foreign Scan on orders_s2 orders_2
```

Evidence 4: Portfolio 5 replication, failover and sharding